JavaScript 1.5

Arrays, Transformations & Copying

Print-first interview refresher | structure → contract → identity → ownership

Trace four things:
index present? | mutation?
return value? | shared identity?

1 | Array model: indexed properties + length boundary

Array.isArray works across realms; instanceof may not.

`["a", , "c"]` length = 3 → write index 4 `length → 5` → shrink length to 1 delete indices ≥ 1

ordinary key `arr.note / arr[-1]`

delete arr[i] hole; same length

position	read	i in arr	hasOwn	callback / copy behavior
hole	`undefined`	false*	false	method-specific: skip | visit | preserve | materialize
explicit undefined	`undefined`	true	true	always an existing element

* false when no inherited index. Console output alone does not reveal structure.

2 | Mutating vs copying: read the return contract

Mutates input

`push / unshift → new length` `pop / shift → removed value`
`splice → removed[]` `sort / reverse → same array`

Returns a new shallow container

`slice / concat / spread / Array.from`
`toSorted / toSpliced / toReversed / with`

slice(start, end)

copy range; end excluded
preserves holes | input unchanged

splice(start, count, ...x)

edit input; shift positions
returns removed values

toSpliced(start, count, ...x)

input unchanged; dense result
returns the full updated array

Ownership rule: mutation of a private local result can be fine; surprising mutation of caller-owned input is an API bug.

3 | Choose by result shape

map	one result per visited element
filter	subset; dense output
find	first value | undefined
some	exists? stop on true
every	all? stop on false
forEach	effect; always returns undefined
reduce	many → one; explicit initial value

Traps: missing return | filter(Boolean) drops 0/false/""
async callbacks are not awaited | readable loop may be clearer

4 | Comparator contract

`result < 0` a before b

`result = 0` stable tie

`result > 0` a after b

pure · equal → 0 · anti-symmetric · transitive

Wrong: `(a, b) => a > b` → only 0 / 1

```
b.score - a.score || collator.compare(a.name, b.name)
```

sort mutates | toSorted copies shallowly
stable order is required; algorithm and complexity are not.
defined values → undefined → holes (sort); toSorted is dense.

5 | Destructuring, rest & spread

```
const [a = fallback, , c, ...tail] = input
```

default for undefined / hole; not for null · rest is last

```
[left, right] = [right, left]
```

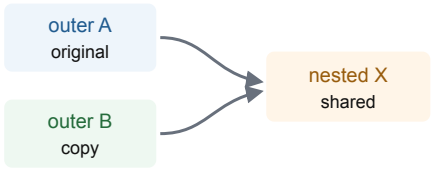
```
[...iterable] fn(...iterable)
```

values via iteration; huge call spread may exceed argument limit

```
{ ...source, key: "later wins" }
```

own enumerable string + Symbol keys; shallow data properties

6 | Shallow copy → structural sharing



New container ≠ deep copy. Shared branches must not be mutated later.
Reference identity supports memoization/change detection; measure copy cost.

7 | Deep copy is a graph contract

cycles · aliases · types · prototypes · descriptors · resources · cost

structuredClone

✓ cycles + repeated aliases
✓ Date · RegExp · Map · Set · buffers
✕ Function · DOM node → DataCloneError
custom prototype / methods / descriptors lost
supported platform types vary by environment

Copy vs transfer

```
clone = structuredClone(value)
```

→ new backing resource

```
{ transfer: [buffer] }
```

original byteLength → 0 (detached)

JSON round-trip traps

cycle / BigInt → TypeError
undefined/function/Symbol → omit or null
NaN/Infinity → null · Date → string
Map/Set → {} · aliases duplicated
Use only for an explicit JSON-compatible schema.

Decision: known model → manual copy | supported graph → structuredClone | transferable resource → explicit ownership move.

Interview trace: presence → length → mutation → return → callback → identity → clone boundary → ownership & measured cost